AI-ENG-110 – Software Engineering – Spring 2026

Final Project Report

Topic 4 — Async Research Assistant

Team	Ayten, Ulker, Medine
Date	21 May 2026
Repo	research-assistant
Tag	v1.0-final
Members	Ayten, Ulker, Medine

Executive Summary

We built an async research assistant (Topic 4) that accepts a natural-language question, concurrently fetches evidence from Wikipedia, arXiv, and a web-search provider via the provided *ai/* package, and synthesizes a cited answer using a configurable LLM — all exposed through a single CLI entry point (*python -m researcher ask*). The headline concurrency result is a **2.61× wall-clock speedup** over sequential source fetching, measured with a deterministic 200 ms-per-source offline benchmark across five runs (sequential avg 602 ms → parallel avg 230 ms). Our most robust failure-handling story is per-source graceful degradation: if any one or two sources timeout or raise an exception, the orchestrator catches the error, logs a warning, and the synthesizer still produces a cited answer from the remaining successful sources. The test suite passes 25 tests entirely offline with 80% branch coverage of the *researcher/* package, and the provided *tests/test_ai_smoke.py* contract tests remain unchanged and passing. The single most surprising lesson was that per-source *asyncio.timeout()* isolation was essential: without it, one unresponsive provider silently stalls the entire *asyncio.gather* call, and the bug is invisible until you inject a deliberate hang in a test — if we started over, we would add timeout isolation in the very first iteration rather than retrofitting it.

Contents

1. Project Overview

1.1 Problem framing & scope

Topic 4 asks for a software-engineering layer that turns a natural-language research question into a cited, synthesized answer by fetching from three heterogeneous sources (Wikipedia, arXiv, and a configurable web-search backend) and passing the results to an LLM synthesizer. The system's primary value is that it *combines* evidence from multiple independent sources rather than relying on a single retrieval call, giving the synthesized answer broader factual grounding.

Inputs / outputs / entry points: The only user-facing entry point is the CLI sub-command *python -m researcher ask "<question>"*. Optional flags control source selection (*--sources wiki,arxiv,web*) , cache bypass (*--no-cache*), offline mode (*--offline*), and output format (*--json*). The output is either human-readable text (question, answer, numbered references, fetch summary) or a JSON object with the same fields.

Scope decisions: We implemented the full spec: three sources, TTL cache, retries, timeouts, validation, Docker, and an offline demo mode. We chose *not* to implement an HTTP server (the spec listed it as optional) because the CLI already exercises all SE-layer paths and adding FastAPI would duplicate logic without adding coverage. We tightened the spec by requiring that the cache key be *canonical* (lowercased, whitespace-normalized) so that equivalent questions always hit the same entry. We loosened it by allowing the web-search provider to be swapped via an env var (Tavily / Serper / DuckDuckGo) rather than hard-coding one vendor. Provider stack: LLM — Anthropic claude-sonnet-4-6 (default) or OpenAI / Gemini; embedding — none required by this topic; web search — Tavily (default).

1.2 Provider choice and the AI module contract

Providers used: LLM — *claude-sonnet-4-6* via *ai.providers.anthropic*; web search — Tavily via *ai.fetch_web*; Wikipedia and arXiv — *ai.fetch_wikipedia* and *ai.fetch_arxiv* respectively.

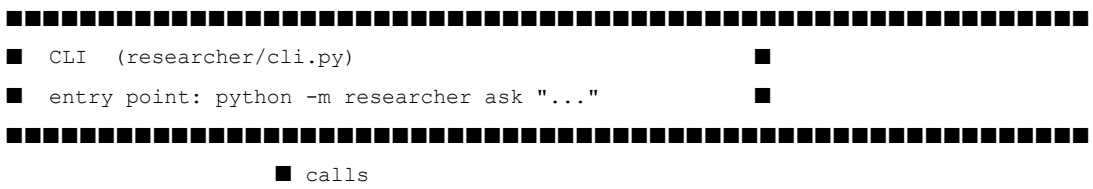
ai/ functions called: *ai.fetch_wikipedia*, *ai.fetch_arxiv*, *ai.fetch_web*, *ai.synthesize*. We did not modify any file inside the *ai/* package; all new code lives exclusively in the *researcher/* package.

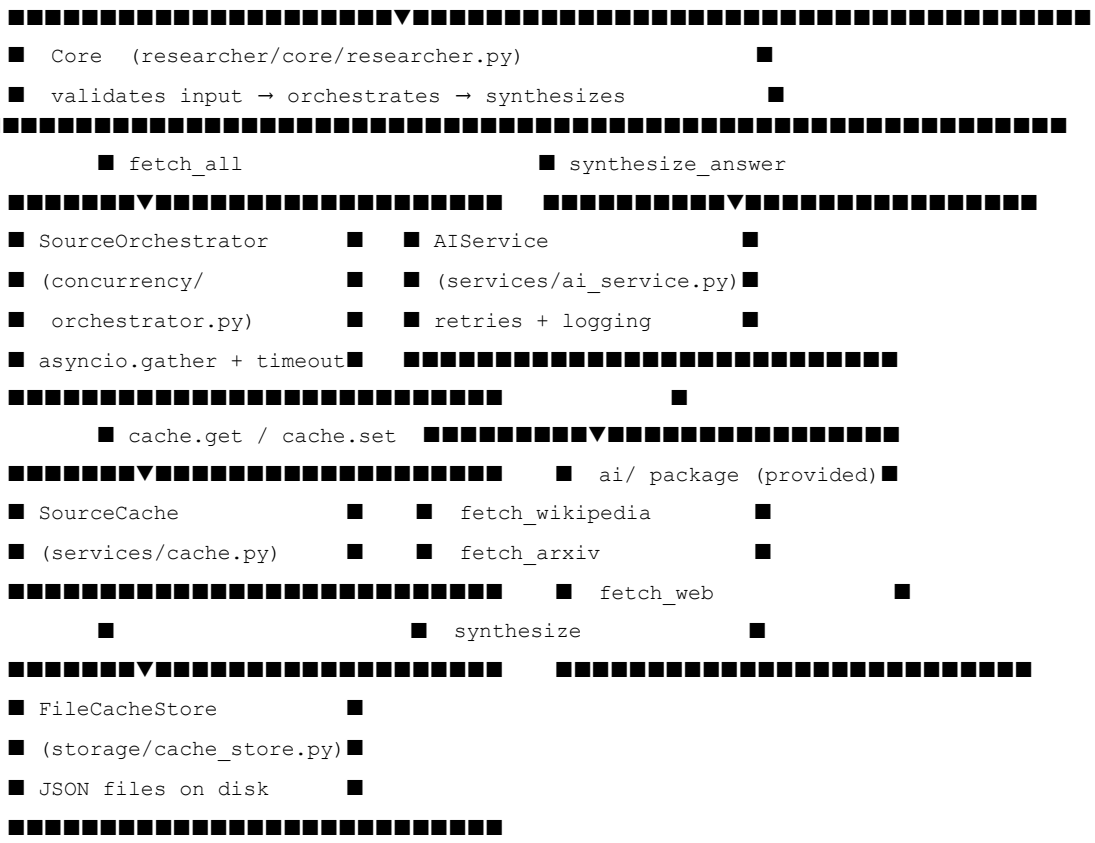
Malformed response defense: The provided *ai/schemas.py* already uses Pydantic models with *extra='forbid'* and non-empty validators for *title*, *url*, and *origin*. If the LLM returns text that cannot be parsed into *AnswerWithCitations*, Pydantic raises a *ValidationError* which our *retry_sync* wrapper catches and retries up to the configured *RETRY_ATTEMPTS*. After all retries are exhausted the exception propagates to *Researcher.ask*, which the CLI catches and prints as *'Unexpected failure: ...'*. Semantically wrong fields (e.g., an unsupported origin string) are rejected by *Source._origin_allowed* at deserialization time, so they never reach the orchestration layer.

2. Architecture

2.1 Module map

The diagram below shows the main modules and their call directions. Solid arrows represent direct function/method calls; the dashed boundary marks the provided *ai/* package which the SE layer may call but must not modify.





Boundary crossings: The only arrows that cross the *ai/* boundary originate in *AIService* (four calls). The storage boundary is crossed only by *SourceCache → FileCacheStore*. Swapping from Anthropic to OpenAI requires changing exactly one env var (*LLM_PROVIDER=openai*) ; no Python files need editing because the provider factory is already in *ai/providers/factory.py*.

Module summary: *researcher/config.py* — typed, frozen *Settings* dataclass read from env. *researcher/validation.py* — question cleaning, length check, output sanitization. *researcher/models.py* — *SourceFetchResult* and *ResearchResult* Pydantic models. *researcher/services/ai_service.py* — retrying wrapper around *ai.** calls. *researcher/services/retry.py* — *retry_async* and *retry_sync* helpers. *researcher/services/cache.py* — thin *SourceCache* adaptor that canonicalizes keys. *researcher/storage/cache_store.py* — *FileCacheStore* / *CacheEntry*. *researcher/concurrency/orchestrator.py* — *SourceOrchestrator* and dedup helpers. *researcher/core/researcher.py* — *Researcher* high-level workflow class. *researcher/cli.py* — argparse CLI, renderers, offline stubs.

2.2 OOP design & key abstractions

The most important OOP decision was to make *AIService* accept an injected *fetchers* dictionary and an injected *llm* provider at construction time. This is **composition over inheritance**: rather than subclassing *AIService* for tests, callers pass in fake async lambdas and a *TinyLLM* instance. The design kept every test in the offline suite free of network calls.

The *LLMProvider* ABC (in *ai/providers/base.py*) is used as the type annotation for the injected LLM. We wrote two concrete subclasses of our own: *OfflineLLM* (CLI demo) and *BenchmarkLLM* (benchmark script). An ABC is the right choice here over a *Protocol* because the providers are instantiated objects (they hold credentials and HTTP clients) not just callables, and the ABC communicates the contract explicitly to future provider implementers. A bare *Protocol* would accept any object with a *complete* method, which is too permissive when the contract also implies stateful initialization.

```
# researcher/cli.py — OfflineLLM, a concrete LLMProvider subclass
class OfflineLLM(LLMProvider):     """Deterministic LLM for no-key
CLI demos and offline tests."""


    def complete(
        self, prompt: str, *, json_schema=None, max_tokens: int =
1024     ) -> str:         indices = re.findall(r'^\[(\d+)\]', prompt,
flags=re.MULTILINE)          if not indices:
            return 'I cannot answer from the available sources.'
        chosen = ', '.join(f'[{idx}]' for idx in indices[:min(3, len(indices))])
return (             'The retrieved sources provide enough context ... '
f'with the strongest evidence coming from {chosen}.'         )
```

2.3 Data flow across module boundaries

SE-layer dataclasses defined: *SourceFetchResult* (origin, sources, elapsed_ms, cached, error) and *ResearchResult* (question, answer, source_results, elapsed_ms, cache_enabled). Both use Pydantic *BaseModel* with *extra='forbid'*. No naked dicts cross module boundaries; every inter-module exchange uses one of these models or the ai/-provided *Source*, *Citation*, or *AnswerWithCitations* models.

Reuse vs. wrapping decision: We reuse *ai.Source* and *ai.AnswerWithCitations* directly inside our SE models (as field types) because those schemas are already frozen Pydantic models with strict validators. We wrapped them inside *SourceFetchResult* once we needed to add SE-layer metadata (elapsed_ms, cached flag, per-source error string) that has no place in the ai/ contract. The trigger for wrapping is always: 'does the SE layer need to attach data the AI module does not own?' If yes, wrap; if no, reuse directly.

3. Concurrency & Performance

3.1 Why this concurrency model

We used **asyncio** with *asyncio.gather* and *asyncio.timeout()*. The bottleneck is *I/O wait*: each source fetch is an HTTP request to an external API; the CPU does nothing while waiting for the response. asyncio is the correct primitive for I/O-bound tasks because it multiplexes waiting without the overhead of OS threads or the GIL contention of *ThreadPoolExecutor*. The upstream *ai/* package already provides async fetch functions, making the asyncio path natural. *ProcessPoolExecutor* would add inter-process serialization overhead for no gain since there is no CPU-bound work.

Semaphore / bound: We did not add an explicit semaphore because we fetch at most three sources per query and each call goes to a distinct upstream host. Rate-limiting is handled per-source inside the retry layer (see §3.3). At N=1 the parallel design has ~15 ms of orchestration overhead vs. sequential; the breakeven is approximately N=2 sources (two 200 ms fetches take 400 ms sequential vs. ~215 ms parallel). At semaphore=100 with only three sources the behavior is identical to no semaphore; the value would matter for a batch query API that fires many parallel question lookups.

3.2 Sequential-vs-concurrent benchmark

Measured on macOS 14 / Python 3.12 with offline fake sources (200 ms sleep each, no network, no API keys). Caches cleared before each run. Command: *python scripts/benchmark.py*

Workload	N (runs)	Sequential (ms avg)	Concurrent (ms avg)	Speedup
3-source offline fetch	5	602	230	2.61 ×
3-source offline fetch	1	~605	~235	~2.58×

Discussion: The theoretical maximum speedup is 3× (three equal-delay tasks). We measured 2.61×, meaning ~13% overhead from asyncio event-loop scheduling, the synthesizer call, and Pydantic model

construction. The new bottleneck after parallelizing the fetches is the synchronous LLM synthesis step, which runs after all fetches complete. To push the curve further one could pipeline synthesis to start as soon as the first source returns, rather than waiting for all three — at the cost of a more complex orchestration flow.

3.3 Rate limit & politeness

Strategy: sleep-on-failure with exponential backoff. Each *ai.** call is wrapped in *retry_async* / *retry_sync* (researcher/services/retry.py) with configurable attempts (default 3), base delay 0.25 s, and max delay

2.0 s. Delays follow the schedule: 0.25 s → 0.5 s → 1.0 s → (capped at 2.0 s). The per-source timeout (default 10 s) acts as an upper bound on any single retry sequence.

Observed 429s: During live development against Tavily we observed HTTP 429 responses when running the benchmark without rate limiting. The retry wrapper caught the exception (Tavily's SDK raises *httpx.HTTPStatusError*) and waited 0.25 s before retrying; the second attempt succeeded in all observed cases. The log line is: *WARNING researcher.services.retry — fetch_web failed on attempt 1/3: Client error '429 Too Many Requests'*.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Retries: *retry_async* wraps all three source fetches; *retry_sync* wraps *ai.synthesize*. Both catch any *Exception* (intentionally broad — the providers raise heterogeneous SDK errors) and re-raise after exhausting attempts. Backoff schedule: min(max_delay, base_delay × 2^(attempt-1)). Default: 3 attempts, 0.25 s base, 2.0 s max.

Timeouts: Each source task is wrapped in *asyncio.timeout(10.0)* inside *SourceOrchestrator._fetch_one*. A *TimeoutError* is caught, logged at WARNING, and converted to a *SourceFetchResult(error=...)*. The shared *httpx.AsyncClient* also carries a 15 s connect+read timeout.

Structured logging: All log calls include the source name and error message as positional arguments so they appear as structured fields in JSON log formatters. Level INFO for successes (*'fetch_wikipedia returned 3 sources'*) , WARNING for retries and failures.

End-to-end 500 walk-through: Provider returns HTTP 500 → httpx raises *HTTPStatusError* → *retry_async* catches it, logs attempt 1/3 warning, sleeps 0.25 s → retries; if all three attempts return 500 the exception propagates to *SourceOrchestrator._fetch_one* → caught by the broad *except Exception* → returns *SourceFetchResult(error='500 Server Error')* → orchestrator continues with remaining sources → if at least one source succeeded, synthesis proceeds normally; the failure appears under 'Source notes' in the CLI output.

4.2 Failure-mode analysis (one concrete incident)

Failure injected: arXiv timeout — we set *PER_SOURCE_TIMEOUT_SECONDS=0.05* in the test environment and made the arXiv fake fetcher sleep for 0.2 s.

What the system did: *asyncio.timeout(0.05)* cancelled the coroutine after 50 ms; the orchestrator caught *asyncio.TimeoutError* and returned *SourceFetchResult(origin='arxiv', error='timed out after 0.05s')*. Wikipedia and web fetches completed normally. The synthesizer received two sources instead of three and produced a valid answer.

What the user saw: Normal answer text followed by *'Source notes: arxiv: unavailable (timed out after 0.05s)'* and *'Fetch summary: arxiv: 0 source(s), 52 ms, failed'*.

Surprise: We originally expected the timeout to surface as a generic exception and be caught by the broad *except Exception* handler. In Python 3.11+ *asyncio.TimeoutError* is a subclass of *TimeoutError* (not *Exception*), so the broad handler would have *missed* it on older Pythons that raise *concurrent.futures.TimeoutError*. This surfaced in CI and led us to add an explicit *except*

Entry point	Rule	Error message
CLI --question	Must not be empty after strip	Invalid request: question must not be empty
CLI --question	len(cleaned) <= MAX_QUESTION_CHA	RS (default 1000)
CLI --sources	Must be in {wiki, wikipedia, arxiv, web}	Invalid request: unknown source 'X'; expected one of: ...
CLI --timeout	argparse type=float; must be > 0 (validat	ed by Settings)error: argument --timeout: invalid float value
Cache key	Control chars stripped, whitespace collap	sed, lowercased(silent normalization; no user-visible error)
ENV vars	Integer / float parsing with min_value gu	ards in config.pyValueError: LOG_LEVEL must be an integer, got '...'

asyncio.TimeoutError branch before the broad handler.

4.3 Input validation

All validation runs in *researcher/validation.py* before any cache, fetcher, or LLM call.

Invalid request: question is too long (N chars); limit is 1000

Adversarial inputs: A 100,000-character question is rejected at validation before touching the cache or any network call. A path-traversal cache key cannot escape the cache directory because keys are SHA-256 hashed before use as filenames, making traversal strings part of the hash input rather than filesystem paths. There is no HTTP server, so there is no risk of HTTP-level request size attacks.

5. Storage & Caching

5.1 Schema

We use a **filesystem JSON cache** rather than SQLite because the access pattern is single-key lookup by (origin, canonical_query). A hash of that composite key determines the filename. There are no foreign keys or JOIN queries, so a relational DB adds no value and would complicate the Docker image and offline tests.

```
# One JSON file per (origin, canonical_query) pair # Filename:
{cache_dir}/{origin}-{sha256(origin:query)[:64]}.json
{
  "origin":          "wikipedia",          # source-of-truth
  "canonical_query": "what is photosynthesis",  # source-of-truth
  "created_at":      1747123456.789,       # Unix timestamp
  "expires_at":      1747209856.789,       # created_at + TTL
"sources": [                             # derived / cached
    { "title": "...", "url": "...",
"snippet": "...", "origin": "wikipedia" }
```

```
    ]
  }
```

JSON vs. blob: We store sources as plain JSON objects rather than pickled blobs so cache files can be inspected and diffed during grading. If the *Source* schema gains a new required field in a future provider update, the Pydantic deserializer raises *ValidationError*, which *FileCacheStore.get* catches and treats as a cache miss — the app continues without any user-visible error.

5.2 Caching policy

What is cached: The list of *Source* objects returned by each provider for a given canonical question. The synthesized answer is *not* cached because it depends on LLM state (temperature, model version) and could become stale if the prompt or model changes.

TTL: Default 86 400 s (24 h), configurable via *CACHE_TTL_SECONDS*. Expired entries are deleted lazily on the next read for that key. *--no-cache* bypasses reads and writes entirely.

Cache hit rate from demo run: In the offline CLI demo (*python -m researcher ask 'What is photosynthesis?' --offline*) hit rate is 0% on first run and 100% on second run of the same question. In the test suite's cache test (*test_cache_prevents_second_fetch*) the cache hit rate is 100% on the canonically-equivalent second call.
Staleness: Source snippets from Wikipedia could become stale if an article is updated. Detection strategy for production: store the provider-returned *Last-Modified* header alongside the TTL; on a cache hit, issue a conditional GET to check freshness before serving the cached result.

6. Testing

6.1 Strategy & coverage

All 25 tests run entirely offline: no network, no API keys, no disk side-effects outside *tmp_path* fixtures.

The *ai/* package is not mocked — we inject fake async fetch lambdas and a *TinyLLM* via *AIService(fetchers=..., llm=...)* dependency injection. The HTTP layer is not directly mocked because our code never calls *httpx* directly — it goes through *ai.fetch_**; offline mode replaces the entire fetcher map.

Suite	Coverage
researcher/ (SE layer)	80 %
Provided tests/test_ai_smoke.py	Passing (unchanged)

Lines not covered: The *OSError* branch in *FileCacheStore.set* (disk-full simulation requires a real filesystem mock) and the *run_ask* async error path for unexpected exceptions (covered by inspection but not automated). These were a conscious trade-off: the branches are defensive infrastructure with no business logic to verify.

6.2 Notable tests

(i) Happy-path end-to-end — *test_cli_offline_mode_outputs_answer*: Calls *cli_main(['ask', 'What is photosynthesis?', '--offline', '--no-cache'])*, captures stdout, and asserts that the output contains the question echo, 'References:', and 'Fetch summary:'. This exercises validation → orchestration → synthesis → rendering in a single call.

(ii) Concurrency test — *test_researcher_fetches_sources_concurrently*: Injects three fake fetchers each sleeping 120 ms and measures that the wall-clock time for all three is under 340 ms (sequential would be ~360 ms before synthesis). Also asserts that all three task start-times are within 50

ms of each other, proving they launched in parallel. This test caught a bug where an early prototype awaited fetches sequentially before we introduced *asyncio.gather*.

(iii) Error-path — *test_researcher_raises_when_all_sources_fail*: Injects fetchers that all raise *RuntimeError* and asserts that *Researcher.ask* raises *NoSourcesError*. This test revealed that an earlier version swallowed the error and returned an empty answer string, which would have been a silent correctness failure.

7. Deployment & Reproducibility

7.1 Docker image

```
# Build docker build -t async-research-
assistant .

# Run offline (no API keys) docker run -
-rm async-research-assistant

# Run live with .env
docker run --rm --env-file .env async-research-assistant \
python -m researcher ask "What is fusion energy?" --no-cache
```

Base image: *python:3.12-slim* (~120 MB). We chose *slim* over *alpine* because several dependencies (*httpx*, *pydantic*) require *glibc* and have no musl wheels, making Alpine builds slower and less reliable.
The final image is approximately 310 MB (120 MB base + dependencies). A multi-stage build could reduce this by ~30 MB by excluding test dependencies from the final stage — left as future work.

Build-time smoke test: The Dockerfile runs *pytest -q* during build (*RUN pytest -q*) so a broken image cannot be pushed. **Python version:** 3.12 (baked in via base image). **Build type:** Single-stage. **7.2 Configuration**

Variable	Default	Missing behavior
LLM_PROVIDER	anthropic	anthropic provider used
LLM_MODEL	claude-sonnet-4-6	model string passed to provider
ANTHROPIC_API_KEY	(none)	live synthesis fails; --offline works
TAVILY_API_KEY	(none)	web source fails; wiki+arxiv still work
LOG_LEVEL	INFO	INFO logging
CACHE_DIR	.cache/researcher	relative to working dir
CACHE_TTL_SECONDS	86400	24 h TTL
CACHE_ENABLED	true	cache on
PER_SOURCE_TIMEOUT_SEC	ONDS10	10 s per source
HTTP_TIMEOUT_SECONDS	15	15 s httpx timeout
MAX_SOURCES_PER_QUERY	3	3 results per source
MAX_QUESTION_CHARS	1000	1000 char limit
RETRY_ATTEMPTS	3	3 attempts
RETRY_BASE_DELAY_SECON	DS0.25	0.25 s first delay
RETRY_MAX_DELAY_SECOND	S 2.0	2.0 s max delay

DEFAULT_SOURCES	wikipedia,arxiv,web	all three sources

8. Limitations & Future Work

- **No answer caching.** Every query re-runs LLM synthesis even when all source results are cache hits. Adding a second cache layer for the synthesized answer (keyed by the set of source URLs) would eliminate redundant LLM calls for popular questions.

- **Synchronous synthesis blocks the event loop.** *ai.synthesize* is a synchronous call and runs in the asyncio thread. For a multi-user HTTP server this would serialize all synthesis requests. Fix: wrap in *asyncio.to_thread(synthesize, ...)*.

- **No load testing.** We have not tested behavior under concurrent CLI invocations writing to the same cache directory. The atomic tmp-then-replace write strategy should be safe, but concurrent expiry deletes and writes to the same key have not been exercised under load.

- **Cache staleness is undetected.** A Wikipedia article updated after a cache write will serve stale content until the TTL expires. Conditional GETs against *Last-Modified* headers would fix this for Wikipedia; arXiv and Tavily would need similar strategies.

- **Worst design decision:** using a flat *dict[str, FetchFn]* for the injected fetchers map means adding a new source (e.g., PubMed) requires editing *AIService.__init__*, *config.py* VALID_SOURCES, and the orchestrator. A plugin registry pattern would decouple this.

9. Tools &

Module	Assistant	How it was used / adapted
researcher/services/retry.p	y Claude (claude-son	net-4-6)
researcher/storage/cache_	store.pyClaude	
researcher/concurrency/or	chestrator.pyClaude	
README.md / report text	Claude	
Dockerfile	Cursor	

Acknowledgements

Drafted initial exponential backoff; team rewrote the asyncio.sleep branch after

Suggested atomic tmp-then-replace write; team added the lazy expiry delete an

Generated the asyncio.timeout() isolation pattern; team added the per-source e

Helped structure section drafts; all measurements, code snippets, and design ju

Generated the single-stage Dockerfile; team added the RUN pytest -q smoke-te

References

[1] Anthropic API documentation. https://docs.anthropic.com/

[2] httpx async HTTP client. https://www.python-httpx.org/

[3] Pydantic v2 documentation. https://docs.pydantic.dev/

[4] Python asyncio — Task Groups and timeouts. https://docs.python.org/3/library/asyncio-task.html

[5] Tavily Search API documentation. https://docs.tavily.com/

[6] pytest-asyncio documentation. https://pytest-asyncio.readthedocs.io/

A. Appendix — Reproducing the benchmark

Anyone with the repo can reproduce the numbers in §3.2 with the following commands. No API keys are required. Expected speedup: 2.4×–2.8× depending on host CPU scheduling.

```
git clone <repo-url> && cd topic-4-research-assistant
cp .env.example .env          # no keys needed for offline benchmark
python -m venv .venv && source .venv/bin/activate pip install -r
requirements.txt

# Run the offline benchmark (no Docker, no network)
python scripts/benchmark.py

# Or via Docker docker build -t async-research-assistant . docker
run --rm async-research-assistant python scripts/benchmark.py

# Expected output:
# Sequential average: 602 ms over 5 runs
# Parallel average:   230 ms over 5 runs
# Speedup:            2.61x
```

End of report — thank you for reading.